

TUGAS PERTEMUAN 11

ALGORITMA DAN STRUKTUR DATA

Nama : Ahmad Rayyan Umar Bawazir

NIM : J0403251135

Kelas : A2

Praktikum 1:

```
J0403251135_AhmadRayyanUmarBawazir.py U × StudiKasusDuniaNyata.py U
J0403251135_AhmadRayyanUmarBawazir_A2 > Praktikum11 > J0403251135_AhmadRayyanUmarBawazir.py > ...
6 # adjacency matrix for the graph:
7 print("Adjacency Matrix for the graph:")
8
9 nodes = [0, 1, 2, 3]
10 edges = [
11     (0, 1),
12     (0, 2),
13     (1, 3),
14     (2, 3),
15 ]
16 # Build adjacency matrix
17 size = len(nodes)
18 adj_matrix = [[0] * size for _ in range(size)]
19 for a, b in edges:
20     adj_matrix[a][b] = 1
21     adj_matrix[b][a] = 1
22
23 print("Adjacency Matrix (4 nodes: 0, 1, 2, 3)")
24 print(" " + " ".join(str(node) for node in nodes))
25 for i, row in enumerate(adj_matrix):
26     print(f"{i}: " + " ".join(str(value) for value in row))
27
28 print("\nRow meanings:")
29 for i, row in enumerate(adj_matrix):
30     connected = [str(j) for j, value in enumerate(row) if value == 1]
31     if connected:
32         print(f"Node {i} terhubung dengan: {' '.join(connected)}")
33     else:
34         print(f"Node {i} tidak terhubung dengan node lain")
```

Hasil Run :

```
Adjacency Matrix for the graph:
Adjacency Matrix (4 nodes: 0, 1, 2, 3)
  0 1 2 3
0: 0 1 1 0
1: 1 0 0 1
2: 1 0 0 1
3: 0 1 1 0

Row meanings:
Node 0 terhubung dengan: 1, 2
Node 1 terhubung dengan: 0, 3
Node 2 terhubung dengan: 0, 3
Node 3 terhubung dengan: 1, 2
```

Praktikum 2:

```
#####  
# Praktikum 11 - Adjacency List  
#####  
print("\nAdjacency List:")  
  
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D'],  
    'C': ['A', 'D'],  
    'D': ['B', 'C']  
}  
  
for node in graph:  
    print(node, "->", graph[node])  
~
```

Hasil Run :

```
Adjacency List:  
A -> ['B', 'C']  
B -> ['A', 'D']  
C -> ['A', 'D']  
D -> ['B', 'C']
```

Praktikum 3 :

```
#####  
# Praktikum 11 - Convert Matrix to List  
#####  
print("\nConvert Adjacency Matrix to Adjacency List:")  
  
matrix = [  
    [0,1,1,0],  
    [1,0,1,0],  
    [1,1,0,1],  
    [0,0,1,0]  
]  
  
adj_list = {}  
  
for i in range(len(matrix)):  
    adj_list[i] = []  
    for j in range(len(matrix[i])):  
        if matrix[i][j] == 1:  
            adj_list[i].append(j)  
  
print(adj_list)
```

Hasil Run:

```
Convert Adjacency Matrix to Adjacency List:  
{0: [1, 2], 1: [0, 2], 2: [0, 1, 3], 3: [2]}
```

Penjelasan

Penjelasan Program:

1. Adjacency Matrix

Program membuat graph menggunakan matrix 4x4.

Angka 1 menunjukkan adanya hubungan antar node, sedangkan 0 berarti tidak ada hubungan.

2. Adjacency List

Graph direpresentasikan dalam bentuk daftar tetangga.

Setiap node menyimpan node lain yang terhubung langsung dengannya.

3. Convert Matrix to List

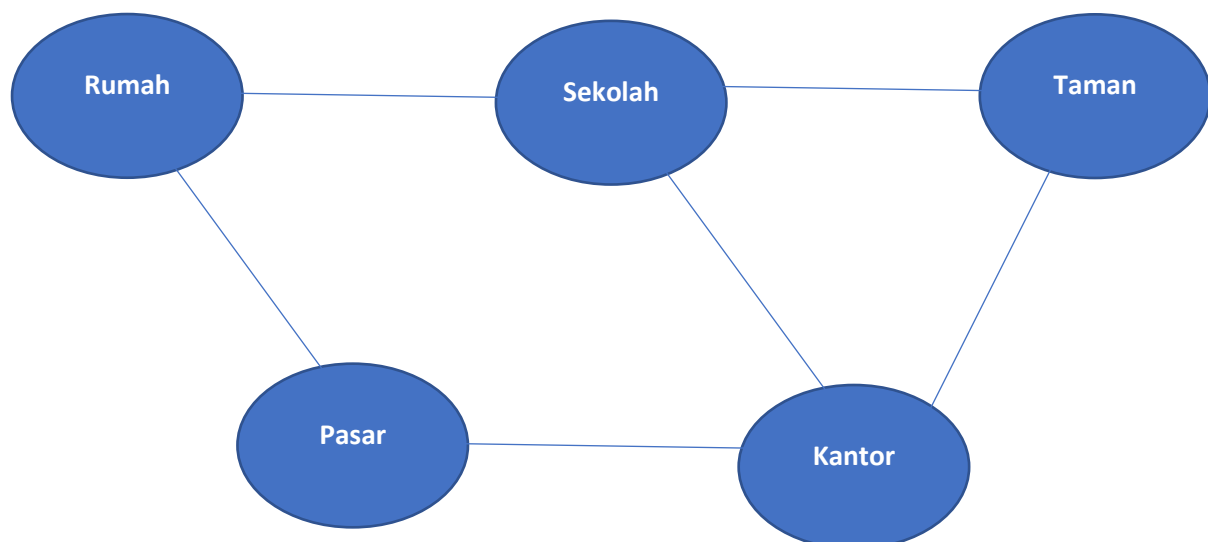
Program mengubah adjacency matrix menjadi adjacency list dengan membaca setiap nilai pada matrix. Jika bernilai 1, maka node dianggap terhubung dan dimasukkan ke adjacency list.

Praktikum 4:

1. Penjelasan Node dan Edge

- **Node (Vertex):** Merepresentasikan lokasi atau titik penting di dalam kota. Dalam kasus ini, kita menggunakan 5 titik: Rumah, Sekolah, Pasar, Taman, dan Kantor.
- **Edge (Sisi):** Merepresentasikan jalan atau jalur yang menghubungkan antar lokasi tersebut. Karena ini peta kota dua arah, maka hubungan antar node bersifat undirected.

2. Gambar Design Graph (Peta Kota)



3. Bayangkan kita sedang mendata **kontak pertemanan** dari 100 mahasiswa di angkatan kita.

- Adjacency Matrix (Si Tabel Kaku)

Ini seperti kita membuat tabel Excel raksasa 100×100 . Barisnya nama mahasiswa, kolomnya juga nama mahasiswa. Kalau si A berteman dengan B, kita kasih angka **1**, kalau tidak berteman kita kasih **0**.

- **Keunggulan:** Sangat cepat kalau mau cek spesifik: "*Eh, si A temenan sama si B gak?*". Kamu tinggal lihat koordinatnya, langsung ketemu jawabannya.
- **Kelemahan: Boros tempat.** Bayangkan kalau si A cuma punya 2 teman, kita tetap harus menyediakan 98 kolom kosong (angka 0) buat dia. Di komputer, ini artinya kita memakan memori RAM untuk menyimpan "ketidakterhubungan" yang sebenarnya tidak penting.

- Adjacency List (Si Daftar Fleksibel)

Ini seperti kita punya buku catatan kecil. Kita tulis nama "Ahmad", lalu di sampingnya kita list siapa saja temannya: "Ivan, Budi". Kita tidak perlu menulis siapa yang *bukan* temannya.

- **Keunggulan: Hemat memori.** Kita cuma mencatat hubungan yang memang ada. Kalau kamu punya 5 lokasi di peta tapi jalannya cuma sedikit (Sparse Graph), cara ini jauh lebih efisien.
- **Kelemahan:** Agak lambat kalau mau cek hubungan spesifik. Kalau mau tahu "*Ahmad temenan sama Citra gak?*", kita harus baca daftar teman Ahmad satu-satu dari awal sampai ketemu (atau tidak ketemu) nama Citra.

4. Kesimpulan

- **Graph sebagai Model Realitas:** Struktur data graph sangat efektif untuk merepresentasikan hubungan antar objek di dunia nyata, baik itu hubungan dua arah (seperti jalan di **Peta Kota**) maupun satu arah (seperti *follower* di **Media Sosial**).
- **Efisiensi Pemilihan Representasi:** Pemilihan antara **Adjacency List** dan **Adjacency Matrix** sangat bergantung pada kepadatan hubungan (*density*) dalam graph.
 - **Adjacency List** lebih unggul untuk *Sparse Graph* (seperti peta kota) karena menghemat penggunaan memori.
 - **Adjacency Matrix** lebih unggul untuk *Dense Graph* karena mempercepat proses pengecekan hubungan antar node secara langsung.
- **Dasar Algoritma Kompleks:** Pemahaman tentang representasi graph (node dan edge) ini merupakan pondasi wajib sebelum mempelajari algoritma penelusuran yang lebih kompleks, seperti **BFS (Breadth First Search)** untuk mencari rute terpendek atau **DFS (Depth First Search)** untuk mengecek keterhubungan seluruh wilayah kota

